



UNIVERSIDADE D
COIMBRA

Complementos de Visão por Computador

Vehicle Counting in Drone-Captured Roundabout
Videos

José Ferreira n2019244998
João Martinho n2020252382

2025/2026

Conteúdo

1	Introdução	2
2	Datasets e Pré-processamento	3
2.1	VisDrone vs. RAIVD	3
2.2	Pré-processamento e Conversão para YOLO	3
3	Deteção de Veículos (Stage 1)	3
3.1	Transfer Learning e Adaptação ao Domínio	3
3.2	Configuração do Treino e Aumento de Dados	3
3.3	Avaliação de Resultados	4
4	Seguimento de Veículos (Stage 2)	4
4.1	Implementação com ByteTrack	4
4.2	Princípios de Funcionamento	4
5	Regiões de Interesse (ROI) e Extração de Métricas (Stage 3)	5
5.1	Definição das Regiões de Interesse (ROI)	5
5.2	Lógica de Monitorização	6
5.3	Visualização em Tempo Real	6
5.4	Identificação Visual e Persistência de Trajetórias	7
6	Resultados e Discussão	7

1 Introdução

A monitorização e análise de tráfego urbano têm sofrido uma transformação significativa desde a introdução de Veículos Aéreos Não Tripulados (UAVs), ou, simplesmente, drones. A *bird's-eye view* proporcionada por estas plataformas oferece uma visão abrangente de infraestruturas críticas, como as rotundas, permitindo a recolha de dados que seriam de difícil obtenção através de câmaras de vigilância fixas ou contagens manuais.

O presente trabalho, realizado no âmbito da unidade curricular de Complementos de Visão por Computador, tem como objetivo o desenvolvimento de um sistema automático capaz de extrair métricas de tráfego a partir de sequências de vídeo capturadas por drones. O foco principal reside na análise de uma rotunda, onde se pretende calcular: a contagem de veículos que entram e saem de cada via (*gates*), as trajetórias individuais com os respetivos tempos de viagem, e o tempo médio de espera nas zonas de aproximação.

Para atingir estes objetivos, foi implementado um *pipeline* de processamento estruturado em três estágios fundamentais. Primeiramente, a deteção de objetos recorrendo ao modelo *Deep Learning* YOLO (*You Only Look Once*), treinado na identificação de veículos. Seguidamente, um estágio de seguimento (*tracking*) multi-objeto utilizando o algoritmo ByteTrack, garantindo a persistência temporal das identidades dos veículos. Por fim, uma camada de análise baseada em Regiões de Interesse (ROI) e lógica geoespacial para a extração das estatísticas de tráfego finais.

Neste relatório, descrevem-se detalhadamente as escolhas metodológicas, os procedimentos de treino realizados nos *datasets* VisDrone e RAIVD, e a avaliação dos resultados obtidos nos vídeos de teste disponibilizado.

2 Datasets e Pré-processamento

A qualidade da detecção de veículos é fortemente influenciada pela semelhança entre os dados de treino e o cenário real de aplicação. Nesta fase inicial, foram exploradas duas fontes de dados distintas: o VisDrone e o RAIVD.

2.1 VisDrone vs. RAIVD

Inicialmente, o modelo foi testado recorrendo ao *dataset* VisDrone. Embora este seja um *benchmark* de referência para drones, os resultados obtidos nos vídeos de teste não foram satisfatórios. A principal razão deve-se à diversidade de ângulos de captura do VisDrone, que inclui vistas oblíquas e horizontais. Como os vídeos de teste do projeto apresentam uma perspetiva estritamente vista de cima, o modelo treinado com o VisDrone apresentou dificuldades em reconhecer a geometria dos veículos a partir do topo, resultando em falhas de detecção e baixos níveis de confiança.

Face a estas limitações, optou-se pela utilização do *dataset* RAIVD (*Roundabout Aerial Imagery Vehicle Dataset*). Este *dataset* revelou-se ideal para a aplicação em questão, uma vez que é focado exclusivamente em imagens de drones sobre rotundas com um ângulo constante de cima. Esta especialização permitiu ao modelo aprender as características visuais específicas que o sistema encontrará na fase de teste.

2.2 Pré-processamento e Conversão para YOLO

O pré-processamento consistiu na adaptação do *dataset* RAIVD para o formato de treino do YOLO. As anotações originais, fornecidas em formato `.csv` com coordenadas absolutas, foram convertidas para ficheiros individuais `.txt`.

Neste processo, as coordenadas foram normalizadas (entre 0 e 1) e transformadas no formato $[Classe, X_{centro}, Y_{centro}, Largura, Altura]$. Esta conversão foi essencial para a compatibilidade com a biblioteca *Ultralytics* utilizada no desenvolvimento do Stage 1.

3 Detecção de Veículos (Stage 1)

Esta primeira fase do sistema consiste na localização e classificação de veículos em cada frame. Para tal, utilizou-se a arquitetura YOLOv8s, selecionada pelo seu equilíbrio entre custo computacional e capacidade de detecção em cenários de pequena escala, típicos de imagens capturadas por drones.

3.1 Transfer Learning e Adaptação ao Domínio

O modelo foi inicializado com pesos pré-treinados no *dataset* COCO, que contém conhecimento genérico sobre formas e objetos. No entanto, para adaptar a rede à *bird's-eye view* das rotundas, realizou-se um processo de *fine-tuning* utilizando o *dataset* RAIVD, como referido anteriormente.

Nesta implementação, optou-se por consolidar todas as categorias originais numa única classe denominada "vehicle". Esta decisão estratégica visa simplificar o problema de contagem e tracking, garantindo que o modelo se foque na morfologia geral do objeto em movimento na rotunda, independentemente da sua tipologia específica, seja ela de um carro, carrinha ou camião.

3.2 Configuração do Treino e Aumento de Dados

A configuração do treino incluiu hiperparâmetros específicos para lidar com os desafios de imagens aéreas:

- **mosaic=0.2:** O aumento de dados por mosaico combina quatro imagens numa só, forçando o modelo a detetar objetos em diferentes contextos e escalas. O valor de 0.2 foi escolhido para introduzir esta diversidade de forma moderada, evitando que a complexidade excessiva da imagem dificultasse a convergência em veículos de dimensões já reduzidas.
- **close_mosaic=10:** Esta funcionalidade desativa o efeito de mosaico nas últimas 10 épocas de treino. A sua utilidade reside na fase de refinamento final, permitindo que o modelo estabilize as suas previsões com base na distribuição real das imagens do RAIVD, sem os artefactos visuais introduzidos pela técnica de mosaico, o que tipicamente resulta num aumento da precisão final.

3.3 Avaliação de Resultados

Para analisar a evolução da aprendizagem, o modelo foi avaliado com diferentes *epochs*. A Tabela 1 apresenta os resultados obtidos para o conjunto de validação.

Treino	Precisão (P)	Recall (R)	mAP ₅₀	mAP _{50–95}
1 Época	0.9179	0.9363	0.9830	0.8246
5 Épocas	0.9968	0.9982	0.9943	0.8783
10 Épocas	0.9973	0.9984	0.9945	0.8922

Tabela 1: Evolução das métricas de desempenho do YOLOv8s ao longo das épocas.

A análise dos dados permite observar a curva de aprendizagem do modelo, onde o incremento do número de épocas contribui para a estabilização do *mAP*, validando a eficácia do *fine-tuning*.

4 Seguimento de Veículos (Stage 2)

Após a deteção dos veículos em cada frame, é necessário estabelecer uma continuidade temporal entre estas instâncias. O seguimento de objetos (*tracking*) permite atribuir um identificador único (ID) a cada veículo, possibilitando a análise da sua trajetória e o cálculo de métricas temporais.

4.1 Implementação com ByteTrack

Neste projeto, optou-se pela utilização do algoritmo ByteTrack. A principal motivação para esta escolha foi a sua compatibilidade nativa com a biblioteca *Ultralytics*, o que simplifica significativamente a implementação através do método `model.track()`. Ao utilizar o parâmetro `persist=True`, o sistema garante que as identidades dos veículos são mantidas mesmo em sequências de vídeo complexas.

4.2 Princípios de Funcionamento

Ao contrário de *trackers* tradicionais que descartam deteções com baixa confiança, o ByteTrack introduz o conceito de "BYTE" (*Bipartite matching*):

1. **Associação de Deteções Fortes:** Primeiro, o algoritmo associa as deteções de alta confiança às trajetórias já existentes.
2. **Recuperação de Deteções Fracas:** Numa segunda etapa, as deteções de baixa confiança (causadas por oclusões parciais ou sombras) são comparadas com as trajetórias que não encontraram par correspondente na primeira fase.

Esta abordagem é particularmente útil no cenário da rotunda analisada, onde obstáculos visuais podem causar quedas momentâneas na confiança do detetor. Graças ao ByteTrack, o sistema consegue "recuperar" o objeto e manter o mesmo ID, evitando a contagem duplicada do mesmo veículo e garantindo a precisão das trajetórias extraídas para o *Stage 3*.

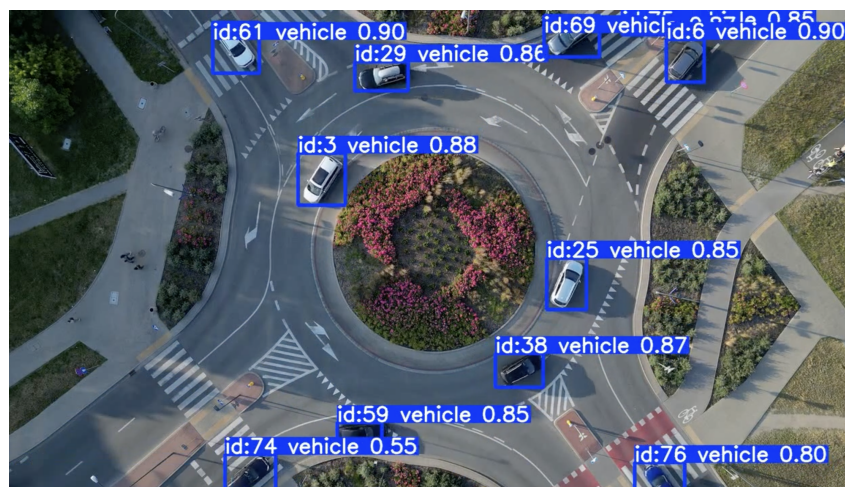


Figura 1: Seguimento de Veículos com IDs com ByteTrack

5 Regiões de Interesse (ROI) e Extração de Métricas (Stage 3)

Neste estágio final, o foco incide sobre o processamento da informação geométrica para converter as trajetórias do *tracker* em dados estatísticos concretos. Esta etapa depende inteiramente da correta definição espacial das zonas de monitorização na rotunda.

5.1 Definição das Regiões de Interesse (ROI)

Para possibilitar a contagem e a medição de tempos, foi necessário definir geograficamente onde começam e terminam as vias de acesso à rotunda. Para o vídeo em análise (*video_005.mp4*), as coordenadas dos pontos foram apontadas manualmente através da observação dos eixos de coordenadas de um frame estático do vídeo.

A escolha da geometria recaiu sobre a forma de um trapézio para cada *gate*, garantindo que toda a largura da via fosse coberta. Estes conjuntos de quatro pontos (x, y) foram organizados num dicionário de dados estruturado (*video_info*), permitindo uma gestão eficiente das configurações para diferentes vídeos.

As ROIs foram categorizadas em dois grupos principais:

- **Gates de Entrada (*Input*):** Identificadas como $i1, i2, i3$ e $i4$, servem como o gatilho inicial para o registo do veículo no sistema.
- **Gates de Saída (*Output*):** Identificadas como $o1, o2, o3$ e $o4$, representam o ponto final da trajetória e o momento em que a contagem de fluxo é validada.

Para facilitar a depuração e garantir o rigor visual, estas coordenadas são sobrepostas dinamicamente sobre o vídeo através da função `cv2.polyline`. Esta visualização permite confirmar que o centroide dos veículos cruza efetivamente as áreas delimitadas antes de serem contabilizados.



Figura 2: Região de interesse

5.2 Lógica de Monitorização

A extração de métricas baseia-se numa máquina de estados gerida pelo dicionário `id_tracker`. Para cada veículo detetado pelo algoritmo *ByteTrack*, o sistema monitoriza a sua progressão através das ROIs utilizando a biblioteca *Shapely* para verificar, através da função `poly.contains()`, se o centroide do veículo se encontra dentro dos polígonos definidos.

A lógica de processamento segue três fases fundamentais:

- **Entrada e Espera:** Quando um veículo é detetado numa *gate* de entrada (i_n), o seu ID é registado e inicia-se o contador de *Wait Time*. Este contador incrementa-se continuamente enquanto o veículo permanecer dentro da geometria da zona de entrada.
- **Percursos :** Assim que o veículo sai da zona de entrada e navega pela rotunda, o sistema inicia a contagem do tempo total de percurso (*Flight Time*).
- **Saída e Validação:** Ao intercepar uma *gate* de saída (o_n), o sistema valida a conclusão da trajetória, atualiza a matriz de contagem de fluxos (`counting_table`) e finaliza os cálculos temporais para esse ID específico.

5.3 Visualização em Tempo Real

Um dos aspetos importantes deste sistema é a integração de um *dashboard* visual dinâmico recorrendo à biblioteca *Matplotlib*. Em vez de apenas processar os dados em *background*, o *script* gera uma figura composta por quatro áreas distintas:

1. **Painel de Vídeo:** Exibe as deteções, IDs e trajetórias sobrepostas.
2. **Tabela de Contagem:** Apresenta o tempo médio gasto, em *frames*, pelos veículos para completar cada trajetória específica.

3. **Tabela de Tempo de Percurso:** Apresenta o tempo médio, em *frames*, gasto pelos veículos para completar cada trajetória específica.
4. **Tabela de Tempo de Espera:** Mostra o tempo médio de retenção, em *frames*, em cada uma das quatro entradas da rotunda.

Esta interface é renderizada frame a frame, convertendo a *canvas* do *Matplotlib* num *array NumPy* para ser gravado no vídeo final via *OpenCV*.

5.4 Identificação Visual e Persistência de Trajetórias

Para garantir a clareza visual no relatório e na demonstração, foi implementado um sistema de cores determinístico baseado no *Track ID*. Através de uma função de *hashing* (*get_color*), cada veículo recebe uma cor única que é mantida durante todo o seu percurso, sendo aplicada tanto à *bounding box* como à linha de trajetória.

A suavização das trajetórias é garantida pelo armazenamento dos últimos 500 pontos centrais de cada ID.

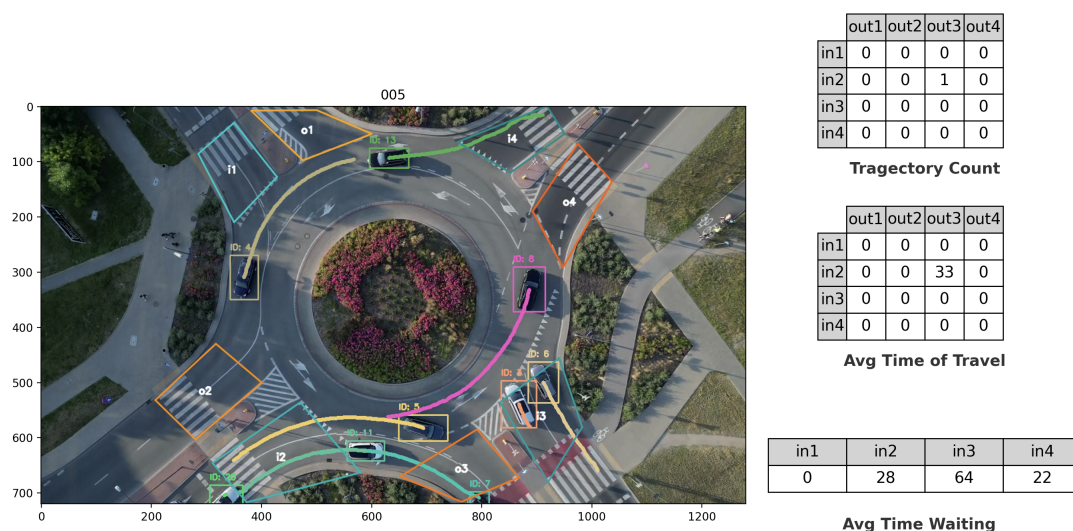


Figura 3: *Dashboard* visual

6 Resultados e Discussão

A análise dos resultados obtidos demonstra que os objetivos definidos para este projeto foram plenamente alcançados. O sistema desenvolvido é capaz de processar sequências de vídeo complexas e extrair dados estatísticos que apresentam uma elevada correspondência com a realidade observada visualmente nos vídeos de teste.

As métricas de fluxo, tempos de percurso e tempos de espera geradas de forma dinâmica no *dashboard* validam a eficácia da integração entre a detecção por *Deep Learning* e a lógica geoespacial baseada em polígonos. Verificou-se que a sensibilidade do sistema às regiões de interesse (ROI) permite uma contabilização precisa dos veículos, mesmo em situações de tráfego moderado.

Contudo, a avaliação técnica permitiu identificar uma limitação no estágio de *tracking*. Em casos isolados, o sistema pode apresentar falhas de persistência, atribuindo dois IDs distintos ao mesmo objeto físico num curto intervalo de tempo. Sendo este um aspeto passível de melhoria em desenvolvimentos futuros do algoritmo.

Para uma demonstração detalhada do sistema em funcionamento, incluindo a renderização das trajetórias coloridas e a atualização das tabelas de métricas em tempo real, foi disponibilizado um vídeo demonstrativo no link abaixo:

Link para o vídeo demonstrativo:

<https://drive.google.com/drive/folders/15uvvKsS58P1vFbib3HD7YtIv1U21v1lC?usp=sharing>

Como resultado final, apresentamos dois exemplos da avaliação do tráfego em rotundas:

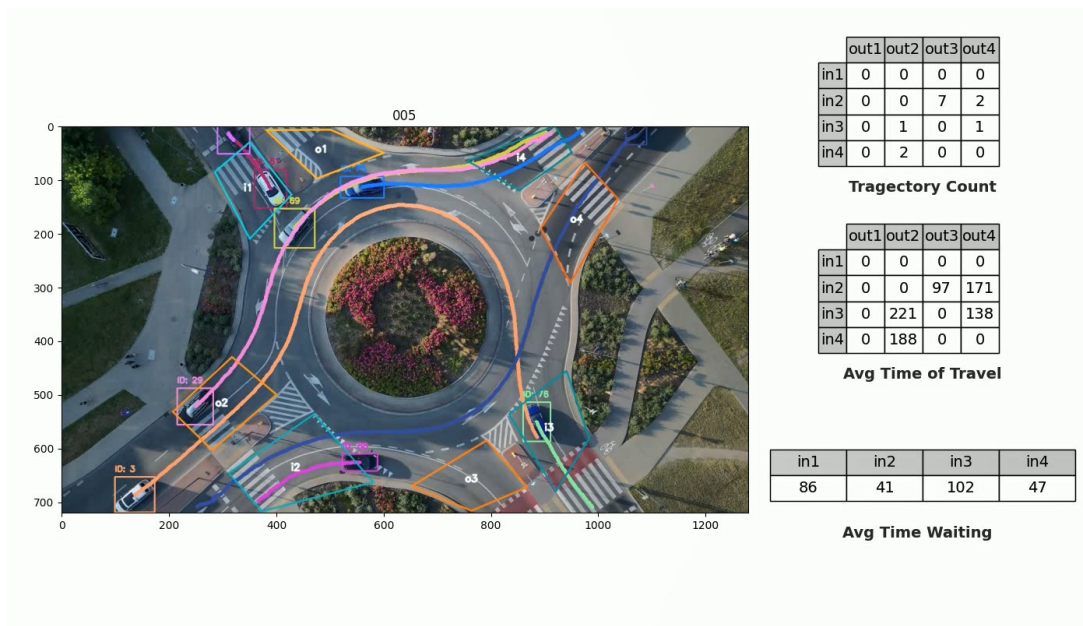


Figura 4: Tráfego avaliado, video_005.mp4

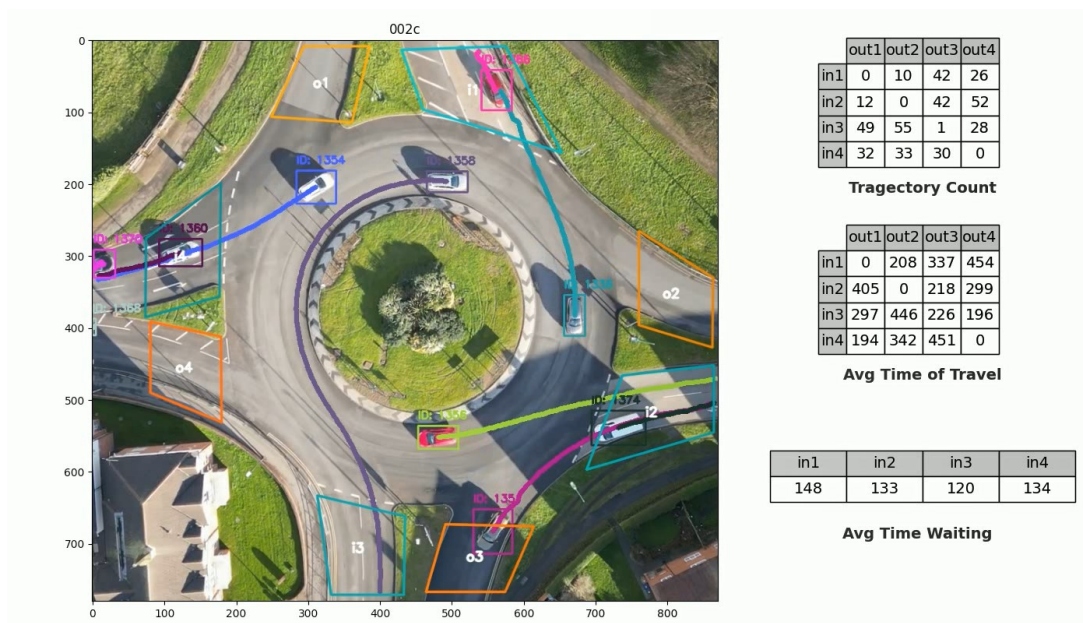


Figura 5: Tráfego avaliado, Video_002_crop.mp4